

AI-Pass

Technical Documentation

Generated 2026-08-02

<https://aipass.space>

This PDF consolidates the AI-Pass technical documentation hub: documentation index, technical overview, developer guide, deployment guide, and auto-models for agents.

Documentation Index

Enterprise AI operating platform: workspace, agents, models, governance, marketplace, and vertical business apps.

Live site: <https://aipass.space>

Start here

Document | Audience | Description

Technical Overview | Engineers, architects | Stack, monorepo layout, runtime rules, key flows

Developer Guide | Contributors | Local setup, scripts, conventions, testing

Deployment | DevOps / release | Static web + Laravel auth deploy to Hostinger

Architecture | Architects | Layered system design

Platform | Product + eng | Module map and OS shell

API | Integrators | REST API surface

Platform foundations

Document | Topic

Runtime Architecture | Planner -> Tool Router -> Execution Engine

AI OS | Operating-system framing of the workspace

Provider / Multi-AI setup | Provider Hub, keys, routing

Auto models | Agent Auto (Standard) default + picker + admin policy

Universal Membership | Free / Professional / Power / Enterprise

Auth - Laravel Auth - Deploy Auth | Identity, sessions, OAuth

Build & orchestration

Document | Topic

Agent Studio | Agent wizard, skills, workflows, execution

LiveSync Engine | Event-driven orchestration

Knowledge Pipeline | RAG, collections, retrieval

AI Governance | AI-system policies, risk, approvals

Trust Engine | Certification and trust scores

Compliance AI | Compliance vertical

Marketplace & store

Document | Topic

Marketplace | Skills registry and publishing

AI Pass Store | End-user app store

Discovery Hub | Search and discovery

Vertical solutions

Document | Topic

Invoice AI | AP automation
Supply Chain AI | Supply chain
Customer Support AI | Support agents
Sales AI | Sales workflows
Content AI | Content generation
Presence Audit | Brand presence across LLMs
ERP Integration | ERP connectors

Operations

Document | Topic
Deployment | Production publish
Backlog | Module status tracking
PHP Auth (legacy) | Legacy PHP auth notes

Quick mental model

```
Browser / Desktop
|
v
apps/web (Next.js static export -> Hostinger public_html)
|
|- /workspace/*      Platform OS UI
|- /api/v1/*         Next route handlers (demo) + Apache proxy
+- /auth/*           -> Laravel auth-api (sessions, Google OAuth, AI proxy)
|
v
packages/*           Domain libraries (pnpm workspace)
|
v
runtime-core -> provider-hub -> wallet -> AI providers
```

Core rule: Application code must not call provider SDKs directly. Route AI through @ai-pass/provider-hub (and credit via @ai-pass/wallet). Agent runs without a pinned model use Auto (Standard) - see Auto models.

Technical Overview

This document is the engineering entry point for the AI-Pass monorepo: what the system is, how packages fit together, and where execution and auth actually run.

Related: [Developer Guide](#) - [Deployment](#) - [Architecture](#) - [Docs index](#)

1. Product definition

AI-Pass is an enterprise AI operating platform, not only an IDE:

- Workspace OS (/workspace) - modules open inside one shell (sidebar + top bar).
- Business builder - requirements -> solutions -> deploy.
- Agent Studio - create, skill, execute, publish agents.
- Model / Provider Hub - multi-provider catalog, routing, membership gates.
- Governance & Trust - policies, approvals, certification.
- Marketplace & Store - skills and vertical apps (Invoice, Support, Supply Chain, ...).
- Auth & billing surface - Laravel sessions + membership tiers + wallet credits.

2. Technology stack

Layer | Choice

Language | TypeScript (apps + packages), PHP 8.4 (Laravel auth-api)

Monorepo | pnpm workspaces + Turbo

Web app | Next.js (App Router), React 19, static export for production

UI | @ai-pass/ui workspace theme (dark OS chrome)

Auth API | Laravel 11 + Socialite (Google), session cookies

AI calls | OpenAI-compatible HTTP (+ Anthropic native path) via Provider Hub / Laravel AiProviderService

Hosting | Hostinger: public_html (static) + laravel-auth (API), Apache .htaccess proxies

Node: >= 20 - pnpm: 9.x

3. Repository layout

```
ai-pass/
|-- apps/
|   |-- web/                # Next.js - marketing + /workspace + /ide
|   |-- desktop/           # Electron shell (legacy / companion)
|   +-- mobile/            # Mobile client
|-- packages/              # Domain libraries (@ai-pass/*)
|-- services/
|   +-- auth-api/          # Laravel - auth, /auth/me, /api/v1/ai/*
|-- php-auth/              # Legacy PHP auth (superseded by Laravel)
|-- scripts/               # build-web-static, deploy-ftp, Laravel deploy
|-- docs/                  # This documentation set
|-- pnpm-workspace.yaml
|-- turbo.json
+-- package.json
```

Packages are published only inside the workspace ("private": true). Web depends on them via "@ai-pass/...": "workspace:*".

4. Architectural rules

4.1 Single AI execution path

```
UI / Agent / Vertical
  -> runtime-core (plan / tools / execute)
  -> provider-hub (catalog, route, health)
  -> ai-core or Laravel AiProviderService
  -> wallet (credits / usage)
```

Do not call OpenAI/Anthropic/Gemini SDKs from apps/web page code.

4.2 Module registry

Workspace modules are declared in @ai-pass/platform-core (PLATFORM_MODULE_DEFS / navigation). Each module has id, route, tier, status, permissions, navOrder.

4.3 Membership ? RBAC

Concern | Package / system
 Billing plan (free/pro/power/enterprise) | @ai-pass/membership
 Model free list / tier gates | config/ai.php + membership
 Agent Auto model lanes | @ai-pass/agent-studio auto-models
 AI-system governance | @ai-pass/governance

5. Runtime data flow (agent execute)

```
sequenceDiagram
  participant UI as Workspace UI
  participant API as Next /api/v1/agents
  participant AS as agent-studio
  participant RC as runtime-core
  participant AM as auto-models
  participant PH as provider-hub

  UI->>API: POST execute + optional modelId
  API->>AS: ExecutionService.execute
  AS->>AM: resolveAgentModel(agent.modelId, override)
  Note over AM: auto -> Standard lane -> concrete model
  AS->>RC: plan + execute
  RC->>PH: route tools / LLM (when wired)
  AS-->>UI: execution + model usedAuto/effectiveModelId
```

See Auto models and Agent Studio.

6. Auth & AI proxy (production)

```
flowchart LR
  Browser --> Apache
  Apache -->|static| PublicHTML[public_html]
  Apache -->|/auth/* /api/v1/ai/*| Laravel[laravel-auth]
  Laravel --> Session[(DB sessions)]
  Laravel --> Providers[OpenAI / Moonshot / ...]
```

Path | Handler

/, /workspace/*, /_next/* | Static Next export

/auth/* | Laravel (login, Google OAuth, /auth/me)

/api/v1/ai/models, /api/v1/ai/chat | Laravel AiProviderService (session auth)

Details: Laravel Auth, Deploy Auth.

7. Package map (selected)

Core platform

Package | Responsibility

platform-core | Modules, nav, org stubs, workspace context

provider-hub | Provider definitions, model catalog, routing

ai-core | Low-level chat/completion adapters

membership | Plans, entitlements, feature gates

wallet | Credits and usage

auth-core | Client auth types / config stubs

ui | Shared React components + workspace theme

shared | Cross-cutting types

Agents & automation

Package | Responsibility

agent-studio | Agents, skills, workflows, execution, auto-models

agent | IDE agent loop / tools

runtime-core | Planner, tool router, execution engine

automation-engine | Workflow graphs

livesync | Live orchestration

mcp | MCP server integration stubs

Knowledge, trust, governance

Package | Responsibility

knowledge-pipeline | RAG / collections

governance | AI governance control plane

trust / trust-engine | Trust scoring

compliance-ai | Compliance vertical

Marketplace & verticals

Package | Responsibility

marketplace* / store* | Skills + store

invoice-ai, supply-chain-ai, customer-support-ai, sales-ai, content-ai | Vertical products

presence-audit | LLM brand presence

erp-connectors / crm-connectors | Enterprise connectors

8. Apps

apps/web

- Marketing site + downloads
- /workspace/* - platform OS
- /ide - in-browser IDE (Monaco, chat, agent)
- Static export via scripts/build-web-static.sh

apps/desktop / apps/mobile

Companion clients; desktop may wrap or deep-link into the web workspace depending on release channel.

services/auth-api

Laravel application: users, sessions, Google OAuth, AI model list + chat proxy, free-tier limits.

9. Configuration surfaces

Config | Purpose

services/auth-api/config/ai.php | Provider keys, model catalog, free_model_ids, endpoints

services/auth-api/.env | Secrets (never commit)

@ai-pass/membership plans | Feature matrix

@ai-pass/agent-studio auto-models | Standard/Premium/Frontier lanes + access policy

packages/platform-core modules | Nav + module metadata

10. Security notes

- Provider API keys stay server-side (Laravel .env / managed secrets).
- Browser uses session cookies for /auth/me and AI proxy.
- SCIM / workspace RBAC (when enabled) assign capabilities via groups, not per-user permission sprawl.
- Rotate any key that has appeared in chat logs or tickets.

11. Related deep dives

- Runtime Architecture
- MULTI-AI-SETUP
- AGENT-STUDIO
- AI-GOVERNANCE
- UNIVERSAL-MEMBERSHIP
- API

Developer Guide

How to work in the AI-Pass monorepo day to day.

See also: Technical Overview - Deployment - Docs index

Prerequisites

- Node.js 20+
- pnpm 9.x (packageManager in root package.json)
- PHP 8.4 + Composer (only if you run Laravel auth-api locally)
- Optional: MySQL matching Hostinger for full auth parity

Install

```
cd /path/to/AI-Pass
pnpm install
```

Build shared libraries you will import (Turbo can orchestrate):

```
pnpm build
# or targeted:
pnpm --filter @ai-pass/shared build
pnpm --filter @ai-pass/platform-core build
pnpm --filter @ai-pass/provider-hub build
pnpm --filter @ai-pass/agent-studio build
pnpm --filter @ai-pass/ui build
```

Run the web app

```
pnpm dev:web
# -> http://localhost:3000
```

URL | Purpose

/ | Marketing

/workspace | Platform OS home

/workspace/playground | Model chat

/workspace/agents | Agent Studio

/workspace/agents/execute | Run agent + model picker

/workspace/agents/settings | Auto-models FAQ + admin allowlists

/ide | In-browser IDE

Without Laravel, many /api/v1/* Next handlers serve demo data. Live AI chat against production keys requires auth-api (see below).

Run Laravel auth-api (local)

```
cd services/auth-api
cp .env.example .env # if needed
composer install
php artisan key:generate
php artisan migrate
php artisan serve --port=8000
```


Point the web proxy / env so /auth and /api/v1/ai reach this process (see Laravel Auth).

Set provider keys in .env (examples):

```
OPENAI_API_KEY=  
OPENROUTER_API_KEY=  
KIMI_API_KEY=  
KIMI_BASE_URL=https://api.moonshot.ai/v1
```

Model catalog: config/ai.php.

Monorepo conventions

Package naming

- npm name: @ai-pass/<name>
- Path: packages/<name>
- "type": "module", build with tsc -> dist/
- Export from src/index.ts

Imports

Prefer package entry points:

```
import { getModels } from '@ai-pass/model-hub';  
import { resolveAgentModel } from '@ai-pass/agent-studio';
```

Do not deep-import another package's src/ unless that package documents it.

AI calls

Apps must go through Provider Hub / Laravel AI proxy - never hardcode vendor SDKs in page components.

Agents & models

- Default / unset model -> Auto (Standard) (auto)
- Override per run via execute modelId
- Documented in AUTO-MODELS.md

UI

Workspace pages use WorkspaceLayoutClient + dark tokens from @ai-pass/ui. Prefer existing patterns over new design systems.

Docs

- Product/engineering docs live in docs/
- Update docs/README.md when adding a new top-level doc

Common scripts

Command | Description

pnpm install | Install workspace

pnpm build | Build all packages/apps (Turbo)
pnpm typecheck | Typecheck
pnpm dev:web | Next dev server
./scripts/build-web-static.sh | Production static export -> apps/web/out
./scripts/deploy-ftp.sh | Upload out/ to Hostinger public_html
./scripts/deploy-hostinger-laravel-auth.sh | Upload Laravel auth-api

Adding a workspace module

1. Add module def in packages/platform-core (route, icon, permissions, nav).
2. Add page under apps/web/app/workspace/<module>/.
3. Wrap with WorkspaceLayoutClient.
4. Document in docs/ and link from docs/README.md.
5. Rebuild platform-core before static export.

Adding a model

1. @ai-pass/provider-hub catalog (+ types if new provider id).
2. @ai-pass/model-hub registry (if Model Hub UI lists it).
3. services/auth-api/config/ai.php models + free_model_ids + endpoints/keys.
4. Optionally AGENT_MODEL_PICKER_OPTIONS in agent-studio/auto-models.ts.
5. Rebuild hubs; redeploy Laravel + web.

Testing tips

- Agent Studio: /workspace/agents/execute with modelId: "auto" and a pinned model.
- Playground: authenticated session required for live /api/v1/ai/chat.
- After static export, confirm CSS hashes exist under apps/web/out/_next/static/css/ before FTP deploy.

Troubleshooting

Symptom | Check

Cannot find module '@ai-pass/...' | pnpm install + build that package's dist/

Playground 401 / empty models | Laravel session + /auth/me + config/ai.php

CSS broken after deploy | Run verify-live-css / ensure full static upload

Agent "model not allowed" | Admin allowlist in Agent Settings / setModelAccessPolicy

Deployment Guide

How AI-Pass is published to production (aipass.space on Hostinger).

See also: Developer Guide - Laravel Auth - Deploy Auth

Production topology

```
Hostinger
|-- domains/aipass.space/public_html/    <- Next static export (apps/web/out)
+-- domains/aipass.space/laravel-auth/   <- Laravel auth-api (outside web root)
```

Apache in public_html proxies:

- /auth/* -> Laravel
- /api/v1/ai/* (and related) -> Laravel

Credentials for FTP scripts live in scripts/.deploy-env.local (gitignored):

```
FTP_HOST=...
FTP_USER=...
FTP_PASS=...
LARAVEL_REMOTE_DIR=/laravel-auth    # optional
FTP_REMOTE_DIR=/                   # optional, public_html root
```

1. Build static web

```
export NODE_OPTIONS=--max-old-space-size=8192
./scripts/build-web-static.sh
```

Output: apps/web/out/

Pre-flight: confirm CSS under apps/web/out/_next/static/css/ and that index.html references those hashes.

Rebuild packages that the web app imports before export (at least):

```
pnpm --filter @ai-pass/platform-core build
pnpm --filter @ai-pass/provider-hub build
pnpm --filter @ai-pass/model-hub build    # if present
pnpm --filter @ai-pass/agent-studio build
pnpm --filter @ai-pass/ui build
pnpm --filter @ai-pass/membership build
```

2. Deploy static site (FTP)

```
./scripts/deploy-ftp.sh
# optional verification:
./scripts/verify-live-css.sh
```

Notes:

- Script uses a lock file (/tmp/aipass-deploy.lock) - do not run two FTP deploys at once.
- Large binaries under /downloads/releases/ can make sync slow; expect long runs.
- Use --force only if you intentionally deploy an export with no CSS references.

3. Deploy Laravel auth-api

```
./scripts/deploy-hostinger-laravel-auth.sh
```

This installs Composer deps targeting PHP 8.4 and uploads the app to LARAVEL_REMOTE_DIR.

On the server after upload (SSH or Hostinger terminal if available):

```
cd ~/domains/aipass.space/laravel-auth
php artisan migrate --force
php artisan config:cache
php artisan route:cache
```

Secrets on server .env

Never commit production secrets. Ensure at least:

Variable | Purpose

APP_KEY | Laravel

APP_URL | https://aipass.space

DB_* | MySQL

GOOGLE_CLIENT_* | OAuth

OPENAI_API_KEY / OPENROUTER_API_KEY / KIMI_API_KEY / ... | AI providers

MAIL_* | Password reset email (not log in production)

Model catalog updates require deploying config/ai.php (via this script) + config:cache clear/rebuild.

4. Post-deploy checks

1. https://aipass.space/ loads with styles (not unstyled HTML).
2. https://aipass.space/auth/me returns JSON (authenticated true/false).
3. Signed-in Playground: GET /api/v1/ai/models lists latest models (incl. Kimi / GPT-5.6 family when configured).
4. Agent Execute: Auto (Standard) resolves and logs the effective model.
5. Google login round-trips to /workspace.

5. Rollback tips

- Keep previous out/ artifact locally; re-upload if a bad static build ships.
- Laravel: redeploy last known-good config/ai.php / code tarball; php artisan config:cache.
- Avoid partial FTP uploads of /_next/static - prefer full deploy script staging flow.

6. Related scripts

Script | Role

scripts/build-web-static.sh | Next static export

scripts/deploy-ftp.sh | Upload web

scripts/verify-live-css.sh | Live CSS hash check

scripts/deploy-hostinger-laravel-auth.sh | Upload Laravel

scripts/deploy-docs-ftp.sh | Optional docs site

7. Environments

Env | Web | Auth

Local | pnpm dev:web :3000 | php artisan serve :8000

Production | Static public_html | laravel-auth behind Apache proxy

Auto Models for Agents

> Part of the technical documentation set.

Agents without a pinned model run on Auto (Standard) - the smart default for the default agent, global agents, and custom agents where no specific model was selected.

Effective: 10 August 2026

FAQ

Which agents use auto models?

The default agent, other global agents and any custom agent you created where no specific model was selected, will run on Standard by August 10.

Will this increase my bill?

No. Your bill will stay absolutely the same and then gradually reduce as we roll out new pricing optimisations in the auto models.

Do I stay in control?

Yes, always. Standard is the smart default, not a lock-in. We are shipping two ways to tailor the model to your task:

1. Model picker in the input bar - switch models on the fly, even with the default agent (/workspace/agents/execute).
2. Admin settings (usage-based pricing plans) - restrict specific groups of models to certain users, user groups, or your whole workspace (/workspace/agents/settings).

Implementation

Piece | Location

Resolution + policy | @ai-pass/agent-studio -> auto-models.ts

Execute routing | ExecutionService + routeAgentModel()

Picker UI | Agent Execute + Agent Wizard

Admin allowlists | Agent Settings -> model access by group

Sentinel ids

- auto / unset -> Standard lane (resolves to gpt-5.6-luna today)
- auto-premium -> Premium lane (gpt-5.6-terra)
- auto-frontier -> Frontier lane (gpt-5.6-sol)
- Any other catalog id -> pinned model

Override at run time

```
POST /api/v1/agents/{id}/execute
{ "input": { "query": "...", "modelId": "kimi-k3" }
```

Omit modelId (or pass auto) to use the agent's configured model / Standard auto.